GML-SHIELD — DAT

Dossier d’architecture technique

Sommaire

1. Contexte et périmètre 2
2. Architecture retenue et justification 3
3. Vue d'ensemble des composants 4
4. Modèle de données et règles métier 5
4.1. Choix de modélisation et protection des données 7
5. Sécurité par couches 9
5.1. Gestion des versions et des correctifs 11
6. Parcours essentiels et diagrammes de séquence simplifiés 12
7. Contrat API REST et documentation 14
7.1. Authentification et gestion des utilisateurs 15
7.2. Gestion des missions 15
7.3. Gestion des justificatifs de livraison 16
7.4. Journal d'audit 16
7.5. Documentation OpenAPI 17
8. Exploitation, qualité et limites 17
8.1. Environnement de production 17
8.2. Qualité du code et validation 18
8.3. Limites et points de vigilance 19
9. Références pour les nouveaux développeurs 21
10. Stratégie Git, intégration continue et livraison 22
10.1. Organisation des branches 22
10.2. Intégration continue 23
10.3. Préparation de la version de production 23
10.4. Déploiement sur le serveur K8 24

1. Contexte et périmètre

GreenMove Logistics (GML) assure des opérations de transport nécessitant la protection d'informations sensibles, notamment les itinéraires, les instructions de livraison et les codes d'accès aux entrepôts.

L'utilisation de canaux de communication grand public, tels que les SMS, les emails non chiffrés ou les applications de messagerie, présente des risques de divulgation et d'accès non autorisé à ces informations.

Pour répondre à ces enjeux, GML a développé **GML-SHIELD**, une application web interne sécurisée permettant de centraliser la gestion et le suivi des missions de transport, ainsi que les échanges d'informations entre répartiteurs et chauffeurs.

La solution repose sur une architecture client/serveur et intègre les fonctionnalités suivantes :

- L'authentification sécurisée et la gestion des utilisateurs selon trois rôles : ADMIN, DISPATCHER et DRIVER.
- La création, l'attribution et le suivi des missions de transport.
- Le chiffrement des codes secrets d'accès aux entrepôts et leur révélation contrôlée à la demande.
- Le téléversement et la consultation sécurisés des preuves de livraison.
- La journalisation des actions sensibles au sein d'un journal d'audit applicatif.
- Les notifications en temps réel lors de l'attribution de nouvelles missions.

L'application est conteneurisée avec Docker et déployée sur un serveur Linux, avec un accès public sécurisé en HTTPS et des communications temps réel en WSS.

Le présent Dossier d'Architecture Technique (DAT) décrit l'architecture actuelle de GML-SHIELD, ses composants, ses flux de données, ses mécanismes de sécurité et les choix techniques ayant guidé sa conception.

2. Architecture retenue et justification

GML-SHIELD repose sur une **architecture monolithique modulaire**, développée avec Laravel/PHP et exposant une API REST consommée par une interface React/TypeScript.

Ce choix permet de centraliser les règles métier, l'authentification et les contrôles de sécurité dans un backend unique, tout en organisant le code en composants distincts et maintenables.

L'utilisation de Laravel simplifie le développement grâce à ses mécanismes intégrés de validation (Form Requests), d'autorisation (Policies), de gestion des transactions et d'accès aux données (Eloquent ORM).

PostgreSQL assure la persistance des données métier, tandis que Laravel Reverb et Laravel Echo permettent la diffusion de notifications en temps réel aux chauffeurs concernés.

L'application est conteneurisée avec Docker Compose et déployée sur le serveur K8, avec des communications sécurisées en HTTPS et WSS.

Cette architecture répond aux besoins de GML en limitant la complexité de développement et d'exploitation, sans introduire la surcharge opérationnelle d'une architecture microservices.

Composant	Décision et justification
React + TypeScript	Interface adaptée aux rôles des utilisateurs ; composants réutilisables, typage statique et facilité de maintenance.
Material UI	Bibliothèque de composants favorisant une interface homogène, responsive et accessible, tout en accélérant le développement.
Laravel / PHP	API REST, validation, autorisations et ORM intégrés ; centralisation de la logique métier dans un backend maintenable.
PostgreSQL	Base relationnelle assurant l'intégrité des données, les contraintes de cohérence et les transactions nécessaires à la gestion des utilisateurs, missions, documents et journaux d'audit.
Sanctum + sessions sécurisées	Authentification de l'application web par session, cookies sécurisés et protection CSRF ; absence de stockage de jetons d'authentification dans le localStorage.
Argon2id	Hachage non réversible des mots de passe avec salage, afin de protéger les identifiants enregistrés en base de données.
Laravel Reverb + Echo	Notifications ciblées sur des canaux privés, autorisations côté serveur, diffusion asynchrone via la file `broadcasts` et actualisation de l'interface React sans rechargement.
OpenAPI / Dedoc Scramble	Documentation OpenAPI 3.1 générée automatiquement à partir du code Laravel, facilitant la compréhension et l'exploration des endpoints.
Docker Compose	Déploiement reproductible des différents services applicatifs, avec séparation des processus, réseaux et volumes persistants.
Reverse proxy Caddy	Point d'entrée HTTPS/WSS de l'application, assurant la terminaison TLS et le routage vers les services frontend, API et WebSocket.

3. Vue d'ensemble des composants

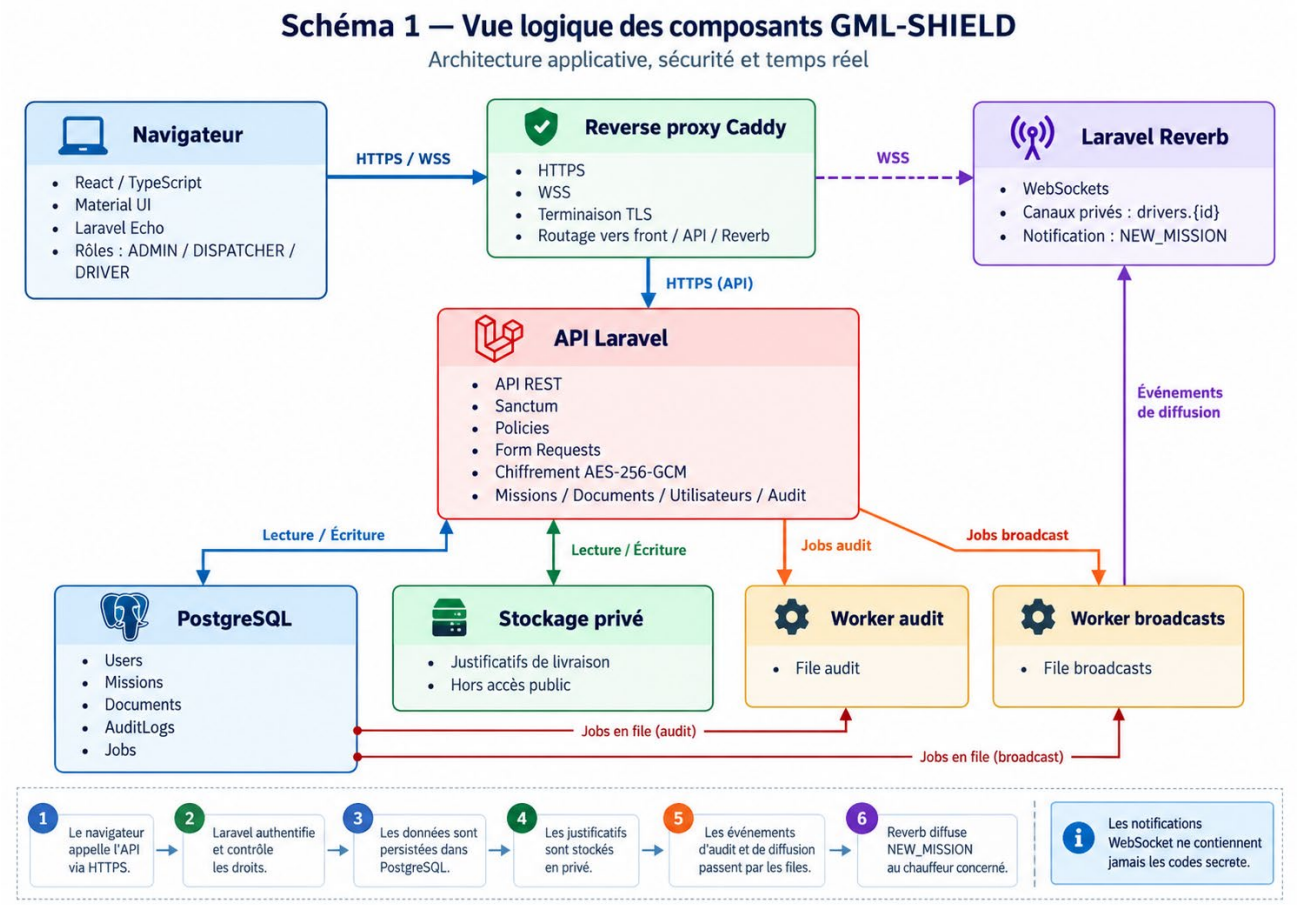
Le schéma suivant présente les principaux composants de GML-SHIELD et leurs interactions.

L'application repose sur une interface React/TypeScript communiquant avec une API REST Laravel. Le backend centralise l'authentification, les autorisations, les règles métier et les accès aux données.

En production, le reverse proxy Caddy sécurise les communications publiques en HTTPS et WSS. PostgreSQL conserve les données applicatives, tandis que Laravel Reverb et les workers assurent les notifications en temps réel et le traitement asynchrone des événements d'audit.

Les codes secrets sont chiffrés en AES-256-GCM avant leur enregistrement, et les justificatifs de livraison sont conservés dans un stockage privé.

Schéma 1 — Vue logique des composants GML-SHIELD



Flux principal de l'application :

1. L'utilisateur accède à GML-SHIELD depuis son navigateur via HTTPS et s'authentifie au moyen de Laravel Sanctum.
2. L'interface React communique avec l'API REST Laravel, qui valide les requêtes et vérifie les autorisations selon le rôle de l'utilisateur.

3. Les missions sont enregistrées dans PostgreSQL, avec chiffrement des codes secrets avant persistance.
4. Lors de la consultation des missions, les codes secrets restent masqués. Le chauffeur assigné peut révéler son code à la demande, uniquement lorsque sa mission est en cours.
5. Les justificatifs de livraison sont téléversés dans un espace privé et consultables après vérification des droits.
6. Les actions sensibles alimentent le journal d'audit applicatif, avec traitement asynchrone par le worker dédié.
7. Lorsqu'une mission est attribuée ou réattribuée à un chauffeur différent, Laravel déclenche l'événement MissionAssigned après validation de la transaction.
8. Le worker broadcasts transmet l'événement à Laravel Reverb, qui diffuse la notification NEW_MISSION sur le canal privé du chauffeur concerné.
9. Laravel Echo reçoit la notification dans l'interface React, qui affiche un message et actualise les missions sans rechargement manuel.

Les notifications WebSocket ne contiennent jamais les codes secrets des missions.

4. Modèle de données et règles métier

Le modèle de données de GML-SHIELD repose sur une base relationnelle PostgreSQL. Il est structuré autour de quatre entités métier principales : User, Mission, Document et AuditLog.

Les relations sont assurées par des clés étrangères, tandis que les autorisations et les règles de gestion sont contrôlées au niveau du backend Laravel.

Entité	Champs principaux et rôle
User	`id`, `name`, `email`, `password`, `role`, `is_active`. Représente les utilisateurs de l'application selon les rôles `ADMIN`, `DISPATCHER` et `DRIVER`. Le champ `password` contient uniquement un hachage Argon2id.
Mission	`id`, `reference`, `title`, `description`, `pickup_address`, `delivery_address`, `scheduled_at`, `secret_code`, `status`, `driver_id`, `created_by`. Représente une mission de transport, son chauffeur assigné et son créateur. Le code secret est chiffré avant stockage.
Document	`id`, `mission_id`, `file_path`, `mime_type`, `uploaded_at`. Représente un justificatif de livraison stocké dans un espace privé et rattaché à une mission.
AuditLog	`id`, `user_id`, `action`, `ip_address`, `timestamp`, `event_id`. Enregistre les opérations sensibles et les événements de sécurité. `user_id` peut être nul pour les actions anonymes et `event_id` est un identifiant UUID unique, nullable, permettant la déduplication.

Diagramme 4 — Modèle relationnel principal de GML-SHIELD



Relations principales :

- Un utilisateur peut créer plusieurs missions, en tant qu'administrateur ou répartiteur.
- Un chauffeur peut être assigné à plusieurs missions.
- Une mission peut posséder plusieurs justificatifs de livraison.
- Un utilisateur peut être associé à plusieurs événements d'audit.
- Les événements d'audit peuvent également être enregistrés sans utilisateur authentifié.

Les clés étrangères contribuent à l'intégrité des données, mais ne remplacent pas les contrôles d'accès applicatifs assurés par les Policies Laravel.

Règles métier principales

RG1 — Cloisonnement des missions

Un chauffeur peut uniquement consulter et gérer les missions qui lui sont assignées. Un répartiteur accède uniquement aux missions qu'il a créées. L'administrateur dispose d'un droit de supervision des missions, sans accès automatique aux codes secrets ni aux justificatifs sensibles.

RG2 — Confidentialité des codes secrets

Les codes d'accès aux entrepôts sont chiffrés avec AES-256-GCM avant leur enregistrement dans PostgreSQL. La clé de chiffrement est conservée séparément des données et hors du dépôt Git.

La lecture habituelle des missions ne divulgue jamais ces codes. Leur révélation est soumise à une autorisation spécifique.

RG3 — Cycle de vie d'une mission

Le parcours nominal d'une mission suit les transitions :

PENDING → IN_PROGRESS → COMPLETED

Un état exceptionnel CANCELLED permet l'annulation contrôlée d'une mission en attente ou en cours, par un administrateur ou le répartiteur propriétaire. Une mission terminée ou annulée ne peut plus poursuivre son cycle de vie.

RG4 — Preuves de livraison

Seul le chauffeur assigné peut téléverser un justificatif de livraison, lorsque sa mission est en cours (IN_PROGRESS).

Au moins un justificatif doit être enregistré avant de permettre la transition vers COMPLETED. Le téléchargement des justificatifs est réservé au répartiteur ayant créé la mission, après vérification de ses autorisations.

4.1. Choix de modélisation et protection des données

Gestion des mots de passe

La colonne users.password conserve le hachage non réversible du mot de passe, réalisé avec Argon2id. Le mot de passe en clair n'est jamais conservé dans la base de données.

Adresses des missions

Les informations de transport distinguent pickup_address, correspondant à l'adresse d'enlèvement, et delivery_address, correspondant à la destination de livraison. Cette séparation permet de représenter plus précisément les opérations logistiques.

Créateur de la mission

Le champ missions.created_by référence l'utilisateur ayant créé la mission. Il peut s'agir d'un DISPATCHER ou d'un ADMIN.

Cette organisation permet aux administrateurs d'assurer la supervision et la continuité opérationnelle, tout en maintenant des contrôles d'autorisation spécifiques selon les actions.

Gestion des statuts

Les statuts sont définis dans l'énumération MissionStatus afin de centraliser les valeurs autorisées et d'éviter leur dispersion dans le code.

Le parcours nominal comprend PENDING, IN_PROGRESS et COMPLETED. Le statut CANCELLED constitue une sortie exceptionnelle permettant l'annulation contrôlée d'une mission non clôturée.

Les transitions sont vérifiées côté serveur, notamment dans MissionPolicy, afin d'empêcher les modifications non autorisées.

Traçabilité applicative

L'entité AuditLog conserve les informations nécessaires à la traçabilité des événements sensibles : utilisateur concerné, action, adresse IP et horodatage.

Le champ event_id, de type UUID et soumis à une contrainte d'unicité lorsqu'il est renseigné, permet de limiter les enregistrements en double lors du traitement asynchrone des événements.

Ce dispositif constitue un journal d'audit applicatif. Il ne garantit pas à lui seul l'inaltérabilité des événements enregistrés.

Révélation contrôlée des codes secrets

Les codes d'accès aux entrepôts sont volontairement exclus des réponses habituelles de consultation des missions.

Leur révélation s'effectue exclusivement par l'endpoint :

GET /api/missions/{mission}/secret-code

Cette opération est autorisée uniquement lorsque les conditions suivantes sont réunies :

- L'utilisateur possède le rôle DRIVER.
- Il est le chauffeur assigné à la mission.
- La mission se trouve au statut IN_PROGRESS.
- La demande est explicitement effectuée par l'utilisateur.

Cette décision répond au principe de moindre privilège : un chauffeur ne doit pas connaître le code d'accès d'un entrepôt avant d'en avoir un besoin opérationnel.

Le passage de PENDING à IN_PROGRESS matérialise la prise en charge de la mission et ouvre temporairement le droit de révélation.

Une fois la mission terminée ou annulée, ce droit est automatiquement refusé.

Les tentatives de révélation autorisées ou refusées font également l'objet d'une traçabilité applicative, sans journalisation du code secret.

5. Sécurité par couches

La sécurité de GML-SHIELD repose sur plusieurs niveaux de protection complémentaires, afin de garantir la confidentialité des informations de transport, l'intégrité des données et le contrôle des accès.

Transport et communications sécurisées

Les communications publiques utilisent HTTPS/TLS pour l'application et l'API REST, ainsi que WSS pour les notifications WebSocket. Le reverse proxy Caddy assure la terminaison TLS et le routage des requêtes. Les cookies de session sont sécurisés et les connexions HTTP sont redirigées vers HTTPS.

Authentification

L'authentification repose sur Laravel Sanctum et des sessions sécurisées. Les utilisateurs se connectent avec leur adresse email et leur mot de passe, transmis exclusivement par une connexion HTTPS.

Les mots de passe des nouveaux comptes doivent comporter au minimum **12 caractères**, sans obligation particulière de majuscules, minuscules, chiffres ou caractères spéciaux. Cette politique privilégie la longueur et l'utilisation de phrases de passe.

Les mots de passe sont hachés avec Argon2id avant leur stockage. Les tentatives de connexion sont limitées et les comptes désactivés sont refusés grâce au contrôle is_active.

Autorisation

Les accès sont contrôlés côté serveur au moyen des rôles, des Policies Laravel et du filtrage des requêtes.

Chaque utilisateur accède uniquement aux fonctionnalités et aux données autorisées pour son rôle. Les identifiants transmis par le frontend ne sont jamais considérés comme une preuve d'autorisation.

Les canaux privés Laravel Reverb appliquent également des contrôles d'authentification et d'autorisation avant toute souscription.

Confidentialité des codes secrets

Le service SecretCodeEncryptionService assure le chiffrement authentifié des codes d'accès aux entrepôts avec AES-256-GCM.

L'implémentation utilise un nonce aléatoire de 12 octets, un tag d'authentification de 16 octets et un format de stockage versionné v1:nonce:tag:ciphertext. Le nonce, le tag et le texte chiffré sont encodés en Base64 avant leur stockage.

La clé de chiffrement de 256 bits est fournie par la configuration SECRET_CODE_KEY, distincte de la clé Laravel APP_KEY et conservée séparément des données chiffrées.

Le modèle Mission masque systématiquement le champ secret_code dans les réponses ordinaires de l'API.

La révélation du secret est effectuée uniquement à la demande du chauffeur assigné, lorsque la mission est au statut IN_PROGRESS. La réponse utilise Cache-Control: no-store afin de limiter sa conservation.

Les erreurs de déchiffrement ne divulguent jamais la valeur du secret.

Validation des entrées et protection SQL

Les données reçues sont validées côté serveur, notamment au moyen des Form Requests Laravel.

Les accès à la base de données utilisent Eloquent ORM et des requêtes paramétrées afin de limiter les risques d'injection SQL.

Les mécanismes de protection CSRF, le typage des données et les règles de sécurité du frontend contribuent également à réduire les risques liés aux entrées utilisateur et aux attaques XSS.

Sécurité des documents

Les justificatifs de livraison acceptés sont les fichiers PDF, JPEG et PNG, dans la limite de 10 Mo.

Le backend vérifie le type MIME détecté, génère un nom de stockage indépendant du nom fourni par l'utilisateur et conserve les fichiers dans un espace privé, hors de l'arborescence publique.

Le téléversement est réservé au chauffeur assigné pendant l'exécution de sa mission. Le téléchargement est autorisé uniquement au répartiteur créateur, après vérification des droits et de l'appartenance du document à la mission.

En cas d'échec de l'enregistrement en base de données, le fichier précédemment stocké est supprimé afin de limiter les incohérences.

L'application ne dispose pas actuellement d'une analyse antimalware approfondie des fichiers. Le contrôle MIME réduit les risques, mais ne garantit pas l'innocuité du contenu.

Traçabilité et journal d'audit

GML-SHIELD dispose d'un journal d'audit applicatif enregistrant les opérations sensibles et certains événements de sécurité.

Le middleware AuditRequest intercepte les requêtes HTTP modifiant des données (POST, PATCH, PUT, DELETE). Des événements spécifiques permettent également de tracer les échecs de connexion, les révélations autorisées ou refusées de codes secrets et les téléchargements de justificatifs.

Chaque événement contient les informations nécessaires à sa traçabilité : identité de l'utilisateur lorsqu'elle est connue, action, adresse IP, horodatage et identifiant d'événement.

Les mots de passe, codes secrets et contenus des requêtes ne sont jamais intégrés volontairement aux événements d'audit.

Le traitement est asynchrone grâce à Laravel Queues, avec une file audit et un worker Docker dédié. Les événements sont enregistrés dans PostgreSQL.

Un identifiant UUID event_id et une contrainte d'unicité contribuent à éviter les doublons lors de la réexécution d'un même job.

La révélation d'un code secret ou le téléchargement d'un justificatif sont refusés lorsque leur événement d'audit ne peut pas être mis en file. Les opérations courantes conservent leur comportement métier en cas d'échec de mise en file, avec signalement de l'incident dans les journaux techniques.

Limites du dispositif

Le journal d'audit assure une traçabilité applicative, mais ne constitue pas un système de conservation inaltérable.

La mise en file asynchrone ne garantit pas à elle seule l'enregistrement définitif des événements. La supervision des jobs échoués, la conservation sécurisée et les mécanismes de protection contre la modification des journaux constituent des axes de renforcement.

La journalisation automatique concerne principalement les requêtes de modification et certains accès sensibles explicitement instrumentés. Elle ne couvre pas systématiquement toutes les consultations de données.

5.1. Gestion des versions et des correctifs

La sécurité du backend repose également sur l'utilisation de versions maintenues de PHP, Laravel et de leurs dépendances.

L'environnement de production GML-SHIELD utilise PHP 8.5 avec Laravel 13. Les versions retenues doivent être compatibles entre elles et bénéficier des correctifs de sécurité disponibles.

Les mises à jour sont préparées en vérifiant leur compatibilité avec les extensions PHP, les dépendances Composer et les fonctionnalités applicatives. Elles sont accompagnées de tests automatisés et de vérifications fonctionnelles avant leur déploiement.

Les dépendances PHP et JavaScript doivent faire l'objet de contrôles de sécurité réguliers, notamment à l'aide de composer audit et npm audit, afin d'identifier les vulnérabilités connues. Ces vérifications constituent une mesure recommandée et ne sont pas encore automatisées dans le pipeline d'intégration continue.

Le hachage des mots de passe repose sur Argon2id, conformément aux bonnes pratiques de sécurité. Ses paramètres sont adaptés aux capacités de l'infrastructure et aux recommandations OWASP.

Laravel Sanctum assure la gestion de l'authentification et des sessions, tandis qu'Argon2id protège les mots de passe enregistrés en base de données. Ces mécanismes répondent à des responsabilités distinctes et complémentaires.

Références techniques :

- PHP — Versions maintenues
- Laravel — Documentation officielle
- OWASP — Password Storage Cheat Sheet

6. Parcours essentiels et diagrammes de séquence simplifiés

Cette section présente les principaux parcours fonctionnels de GML-SHIELD, les échanges entre le frontend React et l'API Laravel, ainsi que les contrôles de sécurité appliqués à chaque étape.

Parcours	Séquence et contrôles associés
Authentification	React → POST /api/auth/login → vérification du mot de passe haché avec Argon2id et du statut is_active → création d'une session sécurisée ou refus d'authentification (422). Les échecs sont tracés par l'événement LOGIN_FAILED, sans enregistrer les identifiants dans le journal d'audit.
Création et attribution d'une mission	DISPATCHER ou ADMIN → POST /api/missions → validation des données et chiffrement du code secret → enregistrement dans PostgreSQL. L'attribution est réalisée séparément via PATCH /api/missions/{mission}/assign. Une transaction et un verrouillage lockForUpdate sécurisent l'opération. Si le chauffeur change, l'événement MissionAssigned est diffusé après validation de la transaction, via la file broadcasts, Laravel Reverb et le canal privé du chauffeur.

Consultation d'une mission	Utilisateur authentifié → GET /api/missions → filtrage des résultats selon le rôle et les droits d'accès. Les codes secrets restent masqués. Seul le chauffeur assigné à une mission IN_PROGRESS peut demander explicitement leur révélation via GET /api/missions/{mission}/secret-code. Les révélations et tentatives refusées sont tracées.
Changement de statut	DRIVER assigné → PATCH /api/missions/{mission}/status → vérification de l'autorisation et de la transition demandée. Le passage à IN_PROGRESS nécessite une mission en attente ; le passage à COMPLETED nécessite une mission en cours et au moins un justificatif enregistré. L'annulation est assurée par un endpoint distinct, réservé aux utilisateurs autorisés.
Dépôt d'une preuve de livraison	DRIVER assigné → POST /api/missions/{mission}/documents → vérification des droits, du statut IN_PROGRESS, du format réel et de la taille du fichier → stockage privé → création de l'enregistrement Document.
Téléchargement d'une preuve	DISPATCHER créateur de la mission → GET /api/missions/{mission}/documents/{document} → contrôle des autorisations, de l'appartenance du document à la mission et du chemin de stockage → mise en file de l'événement DELIVERY_PROOF_DOWNLOADED → transmission du fichier. Le téléchargement est refusé si l'événement d'audit ne peut pas être mis en file.

Les contrôles sont exécutés côté backend, indépendamment des restrictions d'affichage du frontend. Les notifications temps réel ne transportent aucune donnée secrète : elles servent uniquement à informer le chauffeur d'une attribution et à déclencher l'actualisation de son interface.

Diagramme 2 — Authentification sécurisée

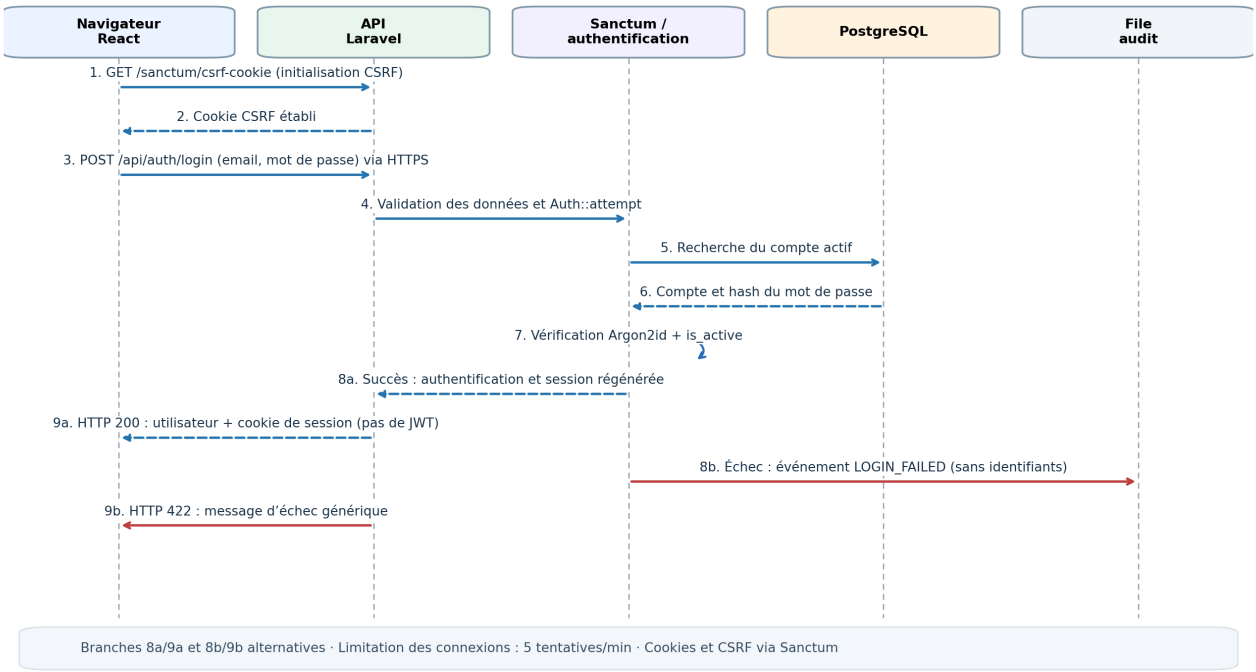
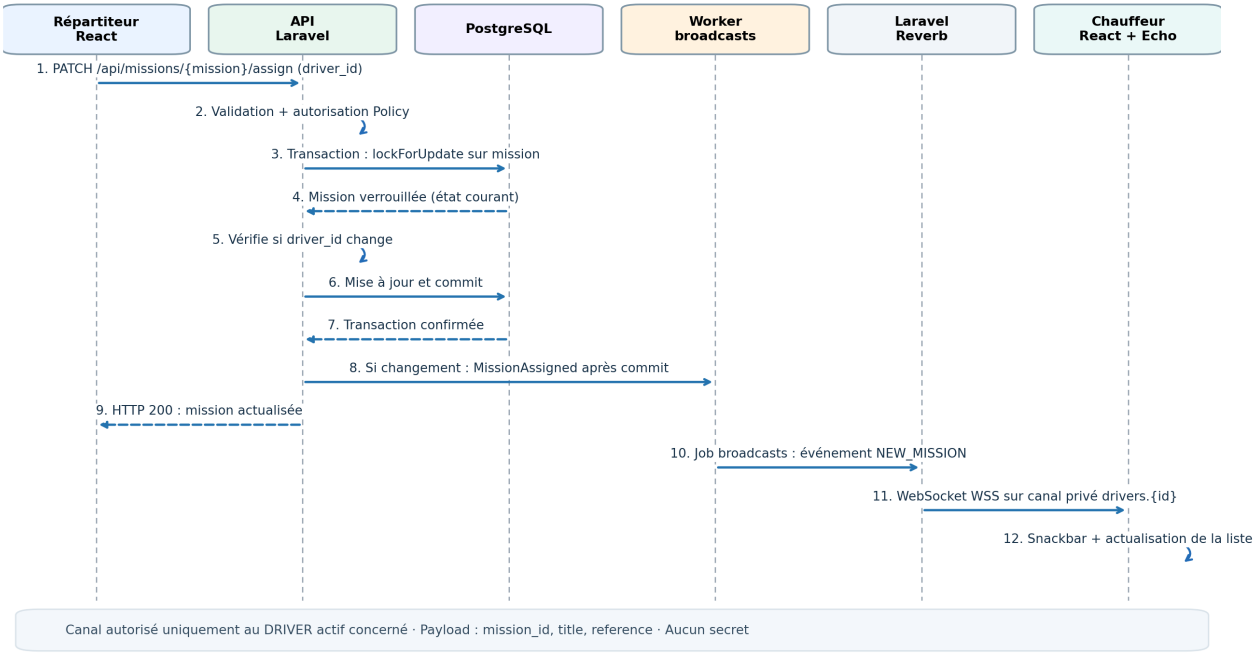


Diagramme 3 — Attribution et notification temps réel



Fiabilité et confidentialité des notifications temps réel

Les notifications sont diffusées uniquement lorsqu'une mission est attribuée à un nouveau chauffeur ou réattribuée à un chauffeur différent. L'événement MissionAssigned est déclenché après validation de la transaction en base de données, afin d'éviter toute notification concernant une modification non confirmée.

Les événements sont traités de manière asynchrone par le worker broadcasts, puis transmis par Laravel Reverb sur le canal privé du chauffeur concerné.

L'accès à ce canal nécessite une authentification valide, un compte actif et le rôle DRIVER. L'identifiant du chauffeur doit correspondre à celui du canal demandé.

La notification NEW_MISSION contient uniquement l'identifiant de la mission, son titre et sa référence. Les codes secrets, les données d'authentification et les informations sensibles de transport ne sont pas diffusés.

Le frontend React utilise Laravel Echo pour recevoir les événements, afficher une notification visuelle et actualiser automatiquement les missions. L'abonnement au canal privé est supprimé lors de la déconnexion ou du changement de compte.

7. Contrat API REST et documentation

GML-SHIELD expose une API REST développée avec Laravel, permettant au frontend React de communiquer avec les services métier.

Les routes applicatives sont protégées par Laravel Sanctum et par un contrôle des comptes actifs. Les autorisations sont ensuite vérifiées selon le rôle de l'utilisateur, la propriété de la mission et l'opération demandée.

Les échanges s'effectuent en HTTPS. Les réponses utilisent le format JSON, à l'exception des téléchargements de documents.

7.1. Authentification et gestion des utilisateurs

Méthode et route	Accès et fonction
POST /api/auth/login	Public : authentification par email et mot de passe, limitation des tentatives et création d'une session sécurisée.
POST /api/logout	Utilisateur authentifié : fermeture de session et renouvellement du jeton CSRF.
GET /api/user	Utilisateur authentifié : récupération des informations du compte connecté.
GET /api/users	ADMIN actif : consultation paginée des utilisateurs.
POST /api/users	ADMIN actif : création de comptes DISPATCHER ou DRIVER, avec validation des données et hachage du mot de passe.
PATCH /api/users/{user}/active	ADMIN actif : activation ou désactivation des comptes DISPATCHER et DRIVER. Les comptes ADMIN sont protégés contre cette modification.
GET /api/drivers	ADMIN ou DISPATCHER autorisé : récupération de la liste des chauffeurs disponibles pour l'attribution des missions.

7.2. Gestion des missions

Méthode et route	Accès et fonction
GET /api/missions	Liste des missions filtrée selon le rôle : toutes les missions pour ADMIN, missions créées pour DISPATCHER, missions assignées pour DRIVER.
GET /api/missions/{mission}	Consultation du détail d'une mission après vérification des autorisations.
POST /api/missions	ADMIN ou DISPATCHER : création d'une mission avec chiffrement du code secret avant son enregistrement.
PATCH /api/missions/{mission}	ADMIN ou DISPATCHER propriétaire : modification d'une mission au statut PENDING.
PATCH /api/missions/{mission}/assign	ADMIN ou DISPATCHER propriétaire : attribution d'un chauffeur à une mission en attente, avec transaction et contrôle des autorisations.
PATCH /api/missions/{mission}/status	DRIVER assigné : démarrage ou clôture de la mission selon les transitions autorisées.

PATCH /api/missions/{mission}/cancel	ADMIN ou DISPATCHER propriétaire : annulation contrôlée d'une mission non clôturée.
GET /api/missions/{mission}/secret-code	DRIVER assigné : révélation explicite du code secret, uniquement lorsque la mission est IN_PROGRESS.

Les codes secrets ne sont jamais inclus dans les réponses ordinaires de consultation des missions.

Lorsqu'une mission est attribuée à un nouveau chauffeur, un événement NEW_MISSION est transmis sur son canal privé via Laravel Reverb, sans divulguer le code secret.

7.3. Gestion des justificatifs de livraison

Méthode et route	Accès et fonction
POST /api/missions/{mission}/documents	DRIVER assigné : téléversement d'un justificatif PDF, JPEG ou PNG de 10 Mo maximum, pendant l'exécution de la mission.
GET /api/missions/{mission}/documents	DISPATCHER créateur : consultation des métadonnées des justificatifs associés à la mission.
GET /api/missions/{mission}/documents/{document}	DISPATCHER créateur : téléchargement sécurisé après vérification des autorisations et de l'appartenance du document à la mission.

Les fichiers sont conservés dans un stockage privé, hors de l'arborescence publique.

La réponse de consultation des missions inclut également l'indicateur has_delivery_proof, permettant à l'interface de déterminer si au moins un justificatif a été enregistré.

La clôture d'une mission est refusée côté serveur tant qu'aucune preuve de livraison n'est disponible.

7.4. Journal d'audit

Méthode et route	Accès et fonction
GET /api/audit-logs	ADMIN actif : consultation paginée des événements enregistrés dans le journal d'audit.

La journalisation des opérations sensibles est réalisée automatiquement par le middleware AuditRequest et par des événements spécifiques, avec traitement asynchrone dans la file audit.

7.5. Documentation OpenAPI

La documentation de l'API est générée automatiquement à partir du code Laravel grâce à Dedoc Scramble, au format OpenAPI 3.1.

Deux points d'accès sont disponibles :

- /docs/api : interface de consultation de la documentation.
- /docs/api.json : spécification OpenAPI au format JSON.

Ces deux routes sont protégées par l'authentification Laravel Sanctum et par l'autorisation viewApiDocs, réservée aux administrateurs actifs.

La documentation facilite la découverte des endpoints, de leurs paramètres et des réponses attendues. Elle constitue une référence complémentaire au présent DAT et doit rester synchronisée avec les évolutions de l'application.

Référence technique : Dedoc Scramble — Documentation officielle

8. Exploitation, qualité et limites

Cette section présente l'environnement d'exploitation de GML-SHIELD, les dispositifs de contrôle qualité et les limites techniques identifiées à ce stade.

8.1. Environnement de production

GML-SHIELD est déployé sur un serveur Linux K8, à l'aide de Docker Compose.

L'application est accessible à l'adresse suivante : https://gml-shield.mkcodecreations.dev

Le reverse proxy Caddy assure les communications sécurisées en HTTPS pour l'application et l'API REST, ainsi qu'en WSS pour les notifications temps réel.

L'environnement de production comprend six services Docker :

Service	Fonction
front	Hébergement de l'interface React compilée et servie par Nginx.
api	Exécution du backend Laravel et exposition de l'API REST.
db	Base de données PostgreSQL, accessible uniquement depuis le réseau Docker privé.
reverb	Serveur WebSocket Laravel Reverb pour les notifications en temps réel.
queue-worker	Traitement asynchrone des événements d'audit.
broadcast-worker	Traitement des événements de diffusion vers Laravel Reverb.

Les données PostgreSQL et les fichiers applicatifs sont conservés dans des volumes Docker persistants. Les secrets de configuration sont fournis par des variables d'environnement non versionnées dans le dépôt Git.

Les migrations Laravel permettent de faire évoluer le schéma de la base de données de manière versionnée.

Le déploiement en production est effectué à l'aide d'une procédure dédiée, après validation du code et préparation des images Docker.

8.2. Qualité du code et validation

La qualité du développement repose sur le guide de qualité de code de GML-SHIELD, qui définit les conventions de programmation, les contrôles attendus et les objectifs de maintenabilité.

Les outils utilisés ou prévus comprennent notamment :

Outil ou contrôle	Utilisation
Laravel Tests / PHPUnit	Vérification automatisée des fonctionnalités backend et des règles de sécurité.
TypeScript	Vérification du typage statique du frontend.
ESLint	Analyse des règles de qualité et de cohérence du code React/TypeScript.
Vite Build	Vérification de la compilation du frontend.
Laravel Pint	Formatage du code PHP selon les conventions retenues.
GitHub Actions	Exécution automatisée des tests backend et des contrôles frontend sur la branche de développement.

Les tests backend, la vérification TypeScript, ESLint et la compilation frontend sont intégrés au workflow d'intégration continue.

Le guide de qualité définit également des objectifs recommandés de couverture des tests d'au moins 80 %, de complexité cyclomatique inférieure ou égale à 10 et de duplication de code inférieure ou égale à 3 %.

Ces indicateurs constituent des références pour l'amélioration continue de la qualité et de la maintenabilité du code. Ils ne sont pas actuellement utilisés comme critères bloquants dans le pipeline d'intégration continue.

Les outils d'analyse complémentaire, notamment PHPStan/Larastan et SonarQube/SonarCloud, sont envisagés pour renforcer progressivement ces contrôles. Leur intégration automatisée et la production de rapports permettant de mesurer ces indicateurs restent à réaliser.

Les validations automatisées sont complétées par des vérifications fonctionnelles de l'application, notamment sur les parcours d'authentification, de gestion des missions, de livraison et de notifications en temps réel.

8.3. Limites et points de vigilance

Malgré son déploiement opérationnel, GML-SHIELD présente plusieurs points nécessitant une attention particulière pour assurer son exploitation durable.

Supervision et disponibilité

L'infrastructure de production GML-SHIELD bénéficie du dispositif de supervision centralisé du serveur K8.

Des rapports automatiques sont transmis sur Discord à 03 h 10 UTC, après les sauvegardes quotidiennes, et à 14 h (heure de Paris). Ils permettent de contrôler notamment l'état des conteneurs Docker, l'utilisation des ressources système, l'espace disque et la cohérence des sauvegardes.

Un contrôle supplémentaire est exécuté toutes les heures afin de détecter les situations critiques, telles qu'un conteneur défaillant, un espace disque insuffisant ou une température excessive. Une alerte Discord est envoyée lors de la détection d'une anomalie, puis lors de son rétablissement.

Ce dispositif complète les mécanismes de redémarrage automatique des conteneurs et permet une surveillance régulière de l'infrastructure.

La surveillance approfondie des traitements applicatifs Laravel, notamment des files d'attente et des événements WebSocket, reste un axe d'amélioration.

Sauvegardes et restauration

Le serveur K8 dispose d'un système de sauvegarde automatisé, exécuté quotidiennement à 03 h UTC sur un disque externe dédié de 4 To.

Le dispositif sauvegarde les fichiers de configuration, les données applicatives, les volumes Docker persistants et les bases de données détectées automatiquement, notamment PostgreSQL.

La politique de conservation prévoit sept sauvegardes quotidiennes et douze sauvegardes mensuelles. Les opérations sont réalisées par étapes indépendantes, avec une nouvelle tentative automatique en cas d'échec. Les résultats sont transmis sur Discord.

Une procédure documentée décrit les opérations de restauration du serveur, de ses conteneurs, de ses volumes et de ses bases de données.

Le contrôle de la restauration effective des données de GML-SHIELD devra néanmoins être validé par un exercice dédié. Une copie supplémentaire hors site ou hors ligne renforcerait également la protection contre la perte simultanée du serveur et de son disque de sauvegarde.

Fiabilité des traitements asynchrones

Les événements d'audit et de notification sont traités par des workers distincts.

La supervision des jobs échoués, leur reprise et la gestion des interruptions de service doivent être renforcées afin d'améliorer la résilience de l'application.

Le temps réel permet l'affichage immédiat des nouvelles attributions lorsqu'un chauffeur est connecté. En revanche, la livraison différée des notifications après une période de déconnexion n'est pas garantie.

Gestion des secrets

Les secrets applicatifs sont séparés du code source et injectés par configuration. Une procédure formalisée de rotation des clés, de renouvellement des secrets et de gestion des incidents reste à définir.

Audit et traçabilité

Le journal d'audit actuel assure une traçabilité applicative des opérations sensibles, sans garantir l'inaltérabilité des enregistrements ni une durée réglementaire de conservation.

La supervision des événements non persistés et la protection renforcée des journaux constituent des points d'amélioration.

Sécurité des documents

Le contrôle des formats, des tailles et des autorisations protège les justificatifs enregistrés. Toutefois, aucune analyse antimalware approfondie ni procédure de quarantaine n'est actuellement intégrée.

Mesure de la qualité et performances

Les contrôles automatisés existants valident les principales fonctionnalités et la compilation du code.

La mesure consolidée de la couverture, l'analyse statique approfondie, la détection de duplication et les tests de charge restent à compléter pour disposer d'indicateurs objectifs de qualité, de performance et de maintenabilité.

Ces limites n'empêchent pas l'utilisation actuelle de GML-SHIELD, mais doivent être prises en compte dans la maintenance, l'exploitation et les évolutions futures de la solution.

9. Références pour les nouveaux développeurs

Cette section regroupe les principales ressources techniques permettant aux développeurs de comprendre l'architecture de GML-SHIELD, d'approfondir les technologies utilisées et d'appliquer les bonnes pratiques de développement et de sécurité.

Backend et authentification

- **Laravel — Documentation officielle :** https://laravel.com/docs
- **Laravel Sanctum — Authentification :** https://laravel.com/docs/sanctum
- **Laravel Broadcasting — Diffusion des événements :** https://laravel.com/docs/broadcasting
- **Laravel Reverb — Communications WebSocket :** https://laravel.com/docs/reverb

Frontend

- **React — Documentation officielle :** https://react.dev/learn
- **TypeScript — Documentation officielle :** https://www.typescriptlang.org/docs/
- **Material UI — Bibliothèque de composants :** https://mui.com/material-ui/

Base de données et infrastructure

- **PostgreSQL — Documentation officielle :** https://www.postgresql.org/docs/
- **Docker Compose — Documentation officielle :** https://docs.docker.com/compose/
- **Caddy — Documentation officielle :** https://caddyserver.com/docs/

Sécurité applicative

- **OWASP Top 10 — Principaux risques de sécurité applicative :** https://owasp.org/www-project-top-ten/
- **OWASP API Security — Sécurité des API :** https://owasp.org/API-Security/
- **OWASP Cheat Sheet Series — Bonnes pratiques de sécurité :** https://cheatsheetseries.owasp.org/
- **NIST SP 800-63B-4 — Digital Identity Guidelines :** https://pages.nist.gov/800-63-4/sp800-63b.html

Recommandations relatives à l'authentification, aux mots de passe, à la vérification des identités et aux mécanismes de sécurité associés.

Documentation de l'API

- **OpenAPI Specification — Standard de description des API :** https://spec.openapis.org/oas/latest.html
- **Dedoc Scramble — Génération de documentation OpenAPI pour Laravel :** https://scramble.dedoc.co/

La documentation OpenAPI propre à GML-SHIELD est également accessible aux administrateurs autorisés depuis l'interface /docs/api.

Ces ressources complètent le présent DAT et le guide de qualité de code. Elles constituent des références pour la maintenance, l'intégration de nouveaux développeurs et les futures évolutions de l'application.

10. Stratégie Git, intégration continue et livraison

GML-SHIELD utilise un dépôt Git unique hébergé sur GitHub, regroupant le backend Laravel, le frontend React et les fichiers de configuration nécessaires au déploiement.

Cette organisation permet de centraliser le suivi des modifications, de conserver un historique commun et de coordonner les évolutions du client et du serveur.

10.1. Organisation des branches

Le projet repose sur plusieurs branches ayant des responsabilités distinctes :

Branche	Rôle
feature/*	Développement isolé d'une fonctionnalité ou d'une correction.
dev	Intégration des développements et exécution des contrôles automatisés.
main	Référence stable du code validé.
release/*	Préparation et revue d'une version destinée à la production.
production	Référence des fichiers applicatifs utilisés pour le déploiement.

Les fonctionnalités sont développées dans des branches spécifiques, généralement créées depuis dev.

Après développement et validation, elles sont intégrées à dev par Pull Request. Une version consolidée peut ensuite être promue vers main.

Cette organisation favorise le travail collaboratif en permettant à plusieurs développeurs d'intervenir simultanément sur des fonctionnalités distinctes, grâce à des branches de travail isolées. Les Pull Requests facilitent les revues de code, les échanges techniques et la détection des conflits avant intégration.

Elle permet également de séparer les développements en cours du code stable et des versions effectivement publiées, tout en assurant la traçabilité des modifications.

10.2. Intégration continue

GitHub Actions assure l'exécution automatisée des principaux contrôles qualité lors des Pull Requests vers dev et des pushes sur cette branche.

Le pipeline comprend :

- Les tests automatisés du backend Laravel.
- La vérification du typage TypeScript.
- L'analyse ESLint du frontend.
- La compilation de l'application React avec Vite.

Ces vérifications permettent de détecter les régressions fonctionnelles, les erreurs de typage et les problèmes de compilation avant l'intégration des développements.

Les contrôles complémentaires recommandés dans le guide de qualité, tels que l'analyse statique approfondie et la mesure de couverture des tests, ne sont pas encore intégrés comme critères bloquants.

10.3. Préparation de la version de production

La publication utilise le script deploy-prod.sh, qui prépare une sélection des composants applicatifs depuis la branche main.

Le processus suit les étapes suivantes :

1. Validation et synchronisation de la branche main.
2. Prévisualisation des fichiers destinés à la production avec ./deploy-prod.sh --dry-run.
3. Préparation de la publication avec ./deploy-prod.sh --publish.
4. Création d'une Pull Request depuis la branche release/* générée vers production.
5. Revue et fusion de la version à publier.

Cette publication sélective permet de ne transférer vers production que les composants nécessaires à l'exécution de l'application, en excluant notamment les fichiers d'environnement contenant des secrets et les dépendances locales.

10.4. Déploiement sur le serveur K8

Après validation de la branche production, le déploiement est réalisé sur le serveur Linux K8 à l'aide de la procédure décrite dans deploy/README-K8.md.

Cette procédure comprend notamment la récupération de la version publiée, la préparation des conteneurs Docker et l'application des migrations nécessaires.

Le redémarrage ou la recréation des services applicatifs est effectué dans le cadre de cette procédure de déploiement.

La publication d'une branche Git ne déclenche donc pas automatiquement la mise à jour des conteneurs de production.

Après déploiement, des contrôles fonctionnels permettent de vérifier l'accessibilité de l'application en HTTPS, l'authentification, les principales opérations métier et les communications WebSocket sécurisées.

Application de production : https://gml-shield.mkcodecreations.dev

La procédure détaillée de publication et d'exploitation est conservée dans le document deploy/README-K8.md du dépôt.